THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Bahawalpur Campus

FINAL YEAR PROJECT PROPOSAL

Smart FAQ's – AI Powered Website Assistant

Nature of Project: Development

Area of Specialization: Web Development, Artificial Intelligence, Natural Language Processing

Submitted By	M. Ahmad Raza (F23BINFT1M01189)
Program	BS Information Technology (BSIT)
Department / Campus	Department of Information Technology — The Islamia University of Bahawalpur
Project Supervisor	Mam Kainat
Academic Session	2023-27

Submitted for academic review and approval

1. Project Information

Project Title	SmartFAQ – AI-Powered Website Assistant
Nature of Project	☒Development ☐Research
Program	☐BSSE ☒BSIT ☐BSAI
Area of Specialization	☒Web Development ☒Artificial Intelligence (AI) ☐Data Science and Analytics

2. Introduction

Websites of educational institutions, businesses, online stores, and service providers receive a steady stream of similar questions from visitors, such as working hours, fees, policies, services, and procedures. The answers usually exist somewhere on the website, in a frequently-asked-questions page, or in a PDF or Word document, but visitors often have to browse several pages or menus to find them, and support staff end up answering the same questions repeatedly.

SmartFAQ is an AI-powered chatbot that can be integrated into any organization's website to answer visitors' questions quickly, accurately, and in a natural, conversational manner. It uses the organization's own website content, frequently asked questions, and uploaded documents as its knowledge sources. Using Natural Language Processing (NLP), LangChain, OpenAI, and ChromaDB, the system understands a visitor's question, retrieves the most relevant information from the organization's knowledge base, and generates an answer based on that information.

The system also includes an administrative dashboard through which website owners can configure the chatbot, upload and manage knowledge sources, and monitor user interactions and feedback. SmartFAQ can be added to an existing website through API or iframe integration, which makes it a flexible and reusable solution for domains such as education, e-commerce, business, and customer support. Its main goal is to improve information accessibility, reduce repetitive inquiries, and enhance the overall user experience.

3. Problem Statement

Visitors who need a quick answer from an organization's website often face a slow and frustrating experience. Static FAQ pages and menu-based navigation require users to know where to look and which keywords to use, while contacting human support may involve waiting. Organizations, in turn, spend staff time on routine questions that are already answered in their existing content.

The project focuses on the following limitations commonly associated with static FAQ pages and manual support:

- Finding an answer may require browsing multiple pages, menus, or long documents, which is time-consuming for visitors.
- Keyword-based search and fixed FAQ lists often fail when users phrase a question in their own words.
- Support staff repeatedly answer the same routine questions, leaving less time for cases that need personal attention.
- Website content, FAQs, and documents change over time, so answers kept in separate places can become outdated or inconsistent.
- Generic AI chatbots that are not grounded in an organization's own content may give incorrect answers, and building a custom chatbot for every website is costly.

Therefore, the proposed system will provide a reusable chatbot that answers questions from an organization's own knowledge sources, can be embedded in any website through API or iframe integration, and gives administrators the tools to manage content and monitor interactions. The proposal identifies a design opportunity; it does not claim that every existing chatbot product has all of the limitations listed above.

4. Project Goal and Objectives

4.1 Overall Goal

To design, develop, and evaluate a web-based AI chatbot that can be integrated into any organization's website and answer visitors' questions accurately and conversationally using the organization's own website content, FAQs, and uploaded documents.

4.2 Specific Objectives

1. Develop a web-based chatbot interface that lets visitors ask questions in natural language and receive relevant answers within a few seconds.
2. Build a knowledge-base pipeline that accepts website content, FAQ entries, and uploaded documents (such as PDF, DOCX, and TXT), splits them into passages, and stores their embeddings in ChromaDB.
3. Implement retrieval-based answer generation using NLP, LangChain, and OpenAI so that responses are grounded in the organization's own content.
4. Provide an administrative dashboard for chatbot configuration, knowledge-source management, and monitoring of user interactions and feedback.
5. Enable integration into existing websites through a REST API and an embeddable iframe widget.
6. Support multiple organizations, each with a separate chatbot, configuration, and knowledge base.
7. Collect user feedback, log queries, and provide a fallback or escalation option when the chatbot cannot find an answer, so that knowledge gaps can be identified.
8. Evaluate answer accuracy and relevance, response time, integration behavior, security, and usability.

5. Proposed System, Main Modules and Scope

Module	Planned functionality
Chat interface (end user)	Embeddable chat widget where visitors type questions, receive conversational answers, give feedback on each response, and see a polite fallback message when no relevant answer exists.
Knowledge base management	Adding website content and FAQ entries, uploading documents (PDF, DOCX, TXT), text extraction, chunking, embedding generation, and storage and updating in ChromaDB.
AI engine (NLP and retrieval)	Understands the visitor's question, retrieves the most relevant passages from ChromaDB, and uses LangChain with an OpenAI model to generate an answer based only on the retrieved content.
Administrative dashboard	Chatbot settings (name, welcome message, appearance), knowledge-source management, conversation and query history, user feedback review, and basic usage analytics.
Organization accounts and chatbot setup	Registration and login for organizations, creation of one or more chatbots, per-organization knowledge bases, and role-based access to the dashboard.
Website integration	REST API with per-chatbot API keys and a ready-to-copy iframe embed code so that the chatbot can be added to an existing website with minimal changes.
Feedback, logging and escalation	Recording of queries, answers, and helpful/not-helpful feedback; identification of unanswered questions; and an option to escalate or leave contact details when no answer is found.

Project scope: the planned deliverable is a demonstration-ready web prototype with the chatbot, knowledge-base management, administrative dashboard, and API/iframe integration, tested on a sample external website. Billing and subscription payments, native mobile applications, voice interaction, production-scale deployment, and deep integration with third-party CRM or helpdesk systems are not assumed unless approved as later extensions.

6. Application Architecture, Technology Stack, Tools and Hardware

The system will use a client–server architecture. The chat widget and administrative dashboard will run in the browser, while a Python backend exposes REST APIs for authentication, chatbot configuration, knowledge ingestion, and question answering. When a visitor asks a question, the backend converts it into an embedding, retrieves the most relevant passages from the organization's ChromaDB collection using LangChain, and sends only the question and those passages to the OpenAI model to generate the answer.

Component	Technology / specification
Frontend	HTML5, CSS3, and JavaScript for the chat widget (iframe-

	embeddable) and the administrative dashboard
Backend / application logic	Python with Django and Django REST Framework for authentication, APIs, document processing, logging, and feedback handling
Artificial intelligence and NLP	LangChain for orchestration; OpenAI language model and embeddings for natural-language understanding and answer generation; retrieval-based (RAG) approach
Knowledge and data storage	ChromaDB as the vector database for knowledge embeddings and metadata; the Django database for accounts, chatbot settings, query logs, and feedback
Integration	REST API with API-key authentication and iframe embed code for external websites
Development tools	Visual Studio Code, Git, GitHub, Postman, browser developer tools
Testing	Manual functional testing, API testing, load testing, and an evaluation set of questions to measure answer quality
Hardware	Laptop/desktop with a modern multi-core processor, 8 GB RAM minimum (16 GB recommended), stable internet, and a current web browser

6.1 High-Level Architecture

USERS Website Visitor (End User) | Organization Administrator | System Administrator
PRESENTATION LAYER Embeddable chat widget (iframe) + administrative dashboard (HTML, CSS, JavaScript)
SERVER-SIDE APPLICATION Authentication • API-key validation • Query processing • Document ingestion • Feedback and query logging • Analytics
AI AND KNOWLEDGE LAYER LangChain | Embeddings | ChromaDB vector search | Separate knowledge collection per organization
EXTERNAL AI SERVICE OpenAI model receives only the visitor's question and the retrieved passages from that organization's knowledge base; if nothing relevant is found, the chatbot says so instead of guessing.

7. Security, Privacy and Data Management

- Protect the administrative dashboard with authentication and role-based access, and verify organization ownership in server-side operations.
- Keep each organization's knowledge base and chatbot data separate so that one organization cannot read or modify another's content.
- Issue API keys per chatbot, store API credentials (including the OpenAI key) in environment variables, and apply rate limiting to the public chat endpoint.
- Validate and sanitize user input and uploaded files (type, size, and content) to reduce risks such as injection, cross-site scripting, and malicious uploads.
- Send only the visitor's question and the retrieved passages to the external AI service, avoid collecting unnecessary personal data, and keep stored logs minimal.
- Make clear that answers are informational and based on the content the organization has provided; instruct the model not to answer beyond the retrieved content.

8. Methodology and Evaluation Plan

The project will follow an iterative development methodology. Requirements will be defined first, followed by architecture and data modelling, implementation of the core modules, integration of the AI pipeline, and testing. Feedback from the supervisor will guide refinements. The project will be evaluated against the stated objectives rather than assuming improvements in support workload or user satisfaction before testing.

Timeline	Activities and expected outputs
Weeks 1–2	Requirements, problem analysis, review of existing FAQ chatbots, and scope confirmation
Week 3	System architecture, data model, API design, user flows, and interface wireframes
Weeks 4–5	Authentication, organization registration, chatbot creation and configuration, and dashboard framework
Weeks 6–7	Knowledge base: FAQ entry, document upload, text extraction, chunking, embeddings, and ChromaDB storage
Weeks 8–9	Question-answering pipeline with LangChain and OpenAI, chat interface, conversation handling, and fallback behavior
Weeks 10–11	REST API, API keys, iframe embed code, and integration testing on a sample external website
Week 12	Feedback collection, query logging, unanswered-question tracking, escalation, and basic analytics
Week 13	Integration, security, performance, and answer-quality evaluation; defect correction
Week 14	Documentation, deployment preparation, final report, and demonstration

8.1 Evaluation Criteria

- Functional testing: verify organization registration, chatbot creation, knowledge upload, chat interaction, feedback, and query logging.
- Answer quality: prepare a set of questions with known answers from the knowledge base; record correct, partially correct, and unsupported answers, and whether the fallback message is shown when no answer exists.
- Integration testing: embed the chatbot in a sample website through iframe and API, and test invalid API keys and rate limits.
- Performance: measure response time and behavior with several concurrent users, with an average response time of about 3 seconds as the target.
- Security and usability: confirm that organizations cannot access each other's data, validate input handling, and run task-based walkthroughs for administrators and end users.

9. Expected Outcomes and Deliverables

- A web-based AI chatbot with a conversational interface for website visitors.
- An administrative dashboard for chatbot configuration, knowledge management, and monitoring of interactions and feedback.
- A knowledge-ingestion pipeline for website content, FAQs, and uploaded documents using embeddings stored in ChromaDB.
- A question-answering pipeline built with NLP, LangChain, and OpenAI that grounds answers in the organization's content.
- API and iframe integration with embed code, demonstrated on a sample external website.
- Feedback collection, query logging, and basic analytics to identify unanswered questions.
- Architecture and data documentation, test cases, and evaluation results.
- Final report and project demonstration.

10. Novelty and Significance

The project does not claim that chatbots, retrieval-augmented generation, or FAQ systems are individually new. Its intended contribution is a practical, reusable package that combines multiple knowledge sources (website content, FAQs, and uploaded documents), grounded AI answers, an administrative dashboard with feedback and query insights, and simple API/iframe integration, so that different organizations can add an assistant to their websites without building one

from scratch. It will also evaluate how accurately the chatbot answers from the supplied content and how well it handles questions it cannot answer.

11. References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/2005.11401
2. LangChain. LangChain Documentation. https://python.langchain.com/docs/
3. Chroma. ChromaDB Documentation. https://docs.trychroma.com/
4. OpenAI. OpenAI API Documentation. https://platform.openai.com/docs
5. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/

12. Project Group Members

Sr. #	Registration No.	Student Name	Phone No.	Signature
1	F23BINFT1M01189	M. Ahmad Raza	_____	_____

13. Approval

Project Supervisor	Mam Kainat
Supervisor Acceptance Signature	_____
Head of Department (HoD)	_____